

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3623 - DATABASE PROGRAMMING 2025/2026 - SEM 1

Project 1 (10%) - Simple Project Management System

Name	:	CHOH JING YI
Matric Id	:	A23CS0296

Name	:	NEO LI XIN
Matric Id	:	A23CS0253

Name	:	CHEW CHUI XIAN
Matric Id	:	A23CS0061



UNIVERSITI TEKNOLOGI MALAYSIA

PRESENTATION SLIDE

Simple Project Management System

Innovating Solutions

www.utm.my

System Overview & Objectives

Background of Domain

A centralized backend database designed to efficiently track projects, assign tasks, and manage employees. By linking detailed records of staff and projects, this system provides clear visibility into project status, employee workloads, and task priorities.

System Objectives

1. Design a normalized database for Employees, Projects, and Tasks.
2. Ensure data integrity using constraints like PRIMARY KEY, FOREIGN KEY, UNIQUE, and CHECK.
3. Implement SQL for all CRUD operations.
4. Perform data retrieval with filtering, joins, and subqueries for reporting.
5. Optimize queries using BTREE and FULLTEXT indexes.

Database Schema (DDL)

```
-- Stores all employee records
CREATE TABLE Employees (
    emp_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    job_title VARCHAR(50),
    hire_date DATE NOT NULL DEFAULT (CURRENT_DATE)
);
```

Employees Table: This table stores employee records and includes a UNIQUE constraint on email. It serves as the "parent" table for project managers and task assignees.

```
-- Projects table
-- Stores all project records
CREATE TABLE Projects (
    project_id INT AUTO_INCREMENT PRIMARY KEY,
    project_name VARCHAR(150) NOT NULL UNIQUE,
    project_description TEXT, -- For FULLTEXT index demo
    start_date DATE,
    due_date DATE,
    manager_id INT, -- This links to an employee

    -- Add FOREIGN KEY constraint
    FOREIGN KEY (manager_id) REFERENCES Employees(emp_id)
);
```

Projects Table: This table stores all project records and includes a UNIQUE constraint on the project name. It utilizes a FOREIGN KEY constraint to assign a manager from the Employees table, ensuring referential integrity.

Database Schema (DDL)

```
-- Tasks table
-- Stores individual tasks, linking projects and employees
CREATE TABLE Tasks (
    task_id INT AUTO_INCREMENT PRIMARY KEY,
    project_id INT NOT NULL,
    assignee_id INT, -- Can be NULL if unassigned
    task_name VARCHAR(200) NOT NULL,
    status VARCHAR(20) DEFAULT 'To Do',
    priority VARCHAR(10) DEFAULT 'Medium',

    -- FOREIGN KEY constraints
    FOREIGN KEY (project_id) REFERENCES Projects(project_id),
    FOREIGN KEY (assignee_id) REFERENCES Employees(emp_id),

    -- CHECK constraints
    CHECK (status IN ('To Do', 'In Progress', 'Done', 'Testing')),
    CHECK (priority IN ('High', 'Medium', 'Low'))
);
```

Tasks Table: This table is the core of the system, linking Projects and Employees. It uses two FOREIGN KEY constraints to ensure referential integrity and CHECK constraints to validate data for status and priority.

Data Manipulation (DML)

Insert Example: This statement adds a new employee to the Employees table.

```
-- Insert at least 10 records into Employees
INSERT INTO Employees (first_name, last_name, email, job_title, hire_date) VALUES
('Chew', 'Chiu Xian', 'ali.hassan@comp.com', 'Project Manager', '2022-01-10'),
('Neo', 'Li Xin', 'beth.tan@comp.com', 'Senior Developer', '2022-03-15'),
('Choh', 'Jing Yi', 'chris.lee@comp.com', 'Junior Developer', '2023-05-20'),
('Lau', 'Ya Kai', 'diana.ravi@comp.com', 'QA Analyst', '2022-07-01'),
('Yap', 'Deh Kai', 'evan.wong@comp.com', 'UI/UX Designer', '2022-09-11'),
('Ramah', 'bin Jaafar', 'fara.ahmed@comp.com', 'Project Manager', '2022-02-14'),
('Chong', 'Joe Lim', 'gani.i@comp.com', 'Database Admin', '2023-01-30'),
('Kristen Chang', 'Jing Huan', 'hana.chen@comp.com', 'Senior Developer', '2022-03-16'),
('Javier', 'Kastilanos', 'ian.gomez@comp.com', 'Junior Developer', '2024-01-10'),
('Jia', 'Li', 'jia.li@comp.com', 'QA Analyst', '2023-08-05'),
('Brendan', 'Chia', 'kumar.nair@comp.com', 'Intern', '2024-06-01'),
('Howard', 'Switsland', 'lena.schmidt@comp.com', 'Business Analyst', '2022-04-10');
```

Data Manipulation (DML)

UPDATE Example: This statement updates a task's status from 'To Do' to 'In Progress' based on its name.

```
-- UPDATE with WHERE
-- Update a task's status from 'To Do' to 'In Progress'
UPDATE Tasks
SET status = 'In Progress'
WHERE task_name = 'Write test cases for login';
```

Data Manipulation (DML)

DELETE Example: This statement removes a cancelled project. We must first delete the tasks associated with it from the Tasks table to avoid violating the foreign key constraint.

```
-- DELETE with WHERE
-- Delete the cancelled project.
-- Must delete tasks from child table first due to foreign key constraints.
DELETE FROM Tasks
WHERE project_id = (SELECT project_id FROM Projects WHERE project_name = 'Cancelled Project Alpha');

DELETE FROM Projects
WHERE project_name = 'Cancelled Project Alpha';
```


Data Query (DQL)

Filtering Example using WHERE, AND, BETWEEN

From the Projects table, the query will showcase the project_name and start_date columns, where the start_date is between 01-01-2024 and 30-06-2024.

```
-- Find all projects starting BETWEEN 2024-01-01 and 2024-06-30 (BETWEEN)
SELECT project_name, start_date
FROM Projects
WHERE start_date BETWEEN '2024-01-01' AND '2024-06-30';
```

DQL operation using WHERE
and BETWEEN condition



	project_name	start_date
▶	E-commerce Platform	2024-01-01
	Mobile Banking App	2024-03-01
	Internal Wiki	2024-02-15
	Data Warehouse Migration	2024-05-01
	Inventory Management Tool	2024-06-15

Result table showcasing all projects between
'2024-01-01' and '2024-06-30'

Data Query (DQL)

Filtering Example using WHERE, NOT

From the Employees table, the query will showcase the first_name, last_name and job_title columns, where the job_title are NOT 'Junior Developer'.

```
-- Find all employees who are NOT 'Junior Developer' (NOT)
SELECT first_name, last_name, job_title
FROM Employees
WHERE NOT job_title = 'Junior Developer';
```

DQL operation using WHERE and NOT condition



	first_name	last_name	job_title
▶	Chew	Chiu Xian	Project Manager
	Neo	Li Xin	Senior Developer
	Lau	Ya Kai	QA Analyst
	Yap	Deh Kai	UI/UX Designer
	Ramah	bin Jaafar	Project Manager
	Chong	Joe Lim	Database Admin
	Kristen Chang	Jing Huan	Senior Developer
	Jia	Li	QA Analyst
	Brendan	Chia	Intern
	Howard	Switsland	Business Analyst

Result table 2.5.2 showcasing employees who are NOT 'Junior Developer'

Data Query (DQL)

Sorting using ORDER BY and LIMIT

From the Employees table, the query will showcase the first_name, last_name and hire_date columns, and with the descending order by their hire_date.

```
-- Get the 5 most recently hired employees (ORDER BY, LIMIT)
SELECT first_name, last_name, hire_date
FROM Employees
ORDER BY hire_date DESC
LIMIT 5;
```

DQL operation using ORDER
BY and LIMIT clause



	first_name	last_name	hire_date
►	Brendan	Chia	2024-06-01
	Javier	Kastilanos	2024-01-10
	Jia	Li	2023-08-05
	Choh	Jing Yi	2023-05-20
	Chong	Joe Lim	2023-01-30

Result table showcasing the 5 most recently hired employees

Data Query (DQL)

Aggregation using COUNT, SUM, AVG

From the Employees table, the SUM function will count the total number of employees.

From the Projects table, the AVERAGE function will take the total_all

```
-- Get the total number of employees, total project budget (SUM), and avg budget (AVG)
SELECT
  COUNT(*) AS total_employees
FROM Employees;

SELECT
  SUM(budget) AS total_all_budgets,
  AVG(budget) AS average_project_budget
FROM Projects;
```

DQL operation using COUNT,
SUM and AVG function



	total_employees
▶	12

	total_all_budgets	average_project_budget
▶	1275000.00	141666.666667

Result table showcasing total employees
and total project budget and average budget

Data Query (DQL)

Aggregation using MIN, MAX function

For the Projects table, the query will get the minimum date and maximum date and showcase them as first_project_start and last_project_date.

```
-- Get the earliest project start date (MIN) and latest due date (MAX)
SELECT
    MIN(start_date) AS first_project_start,
    MAX(due_date) AS last_project_due
FROM Projects;
```

DQL operation using MIN and MAX function



	total_employees
▶	12

	total_all_budgets	average_project_budget
▶	1275000.00	141666.666667

Result table showcasing the earliest project start date and latest due date

Data Query (DQL)

Grouping and filtering groups (GROUP BY, HAVING)

From the Tasks table, the query will showcase project_id and number_of_tasks columns, where number_of_tasks represents the total tasks for each project.

```
-- Show employees who are assigned MORE THAN 2 tasks (GROUP BY, HAVING)
SELECT assignee_id, COUNT(task_id) AS task_count
FROM Tasks
WHERE assignee_id IS NOT NULL
GROUP BY assignee_id
HAVING task_count > 2;
```

DQL operation using GROUP BY and HAVING clause



	assignee_id	task_count
▶	2	4
	5	3
	7	3
	8	4

Result table showcasing employees who are assigned to more than 2 tasks

Data Query (DQL)

Numeric and string functions (ROUND)

From the Projects table, the query displays the average project budget and rounds it to 2 decimal places using the ROUND function.

```
-- Show average project budget, rounded to 2 decimal places (ROUND)
SELECT ROUND(AVG(budget), 2) AS rounded_avg_budget
FROM Projects;
```

DQL operation using ROUND function



	rounded_avg_budget
▶	141666.67

Result table showcasing the average project budget (2 d.p.)

Data Query (DQL)

Numeric and string functions (TRUNCATE)

From the Projects table, the query displays the average project budget and truncates it to zero decimal places with the TRUNCATE function.

```
-- Show average project budget, truncated (TRUNCATE)
SELECT TRUNCATE(AVG(budget), 0) AS truncated_avg_budget
FROM Projects;
```

DQL operation using
TRUNCATE function



	truncated_avg_budget
▶	141666

Result table showcasing the
truncated average project budget
(0 d.p.)

Data Query (DQL)

Numeric and string functions (CONCAT and UPPER)

From the Employees table, the query displays the full_name and job_title columns, where the first_name and last_name are concatenated using the CONCAT function and converted to uppercase using the UPPER function.

```
-- Get employee full names in uppercase (CONCAT, UPPER)
SELECT UPPER(CONCAT(first_name, ' ', last_name)) AS full_name, job_title
FROM Employees;
```

DQL operation using CONCAT and UPPER function



	full_name	job_title
▶	CHEW CHIU XIAN	Project Manager
	NEO LI XIN	Senior Developer
	CHOH JING YI	Junior Developer
	LAU YA KAI	QA Analyst
	YAP DEH KAI	UI/UX Designer
	RAMAH BIN JAAFAR	Project Manager
	CHONG JOE LIM	Database Admin
	KRISTEN CHANG JING HUAN	Senior Developer
	JAVIER KASTILANOS	Junior Developer
	JIA LI	QA Analyst
	BRENDAN CHIA	Intern
	HOWARD SWITSLAND	Business Analyst

Result table showcasing the full name of employees in capital letters

Data Query (DQL)

Numeric and string functions (LENGTH)

From the Projects table, the query displays the project_name and name_length columns, where the length of the project name is calculated using the LENGTH function.

```
-- Get the length of each project's name (LENGTH)
SELECT project_name, LENGTH(project_name) AS name_length
FROM Projects;
```

DQL operation using LENGTH function



	project_name	name_length
▶	AI Chatbot	10
	Customer Feedback System	24
	Data Warehouse Migration	24
	E-commerce Platform	19
	HR Payroll System	17
	Internal Wiki	13
	Inventory Management Tool	25
	Mobile Banking App	18
	Website Redesign	16

Result table showcasing the length of each project's name

Data Query (DQL)

Conditional logic (CASE WHEN)

From the Tasks table, the query displays the task_name, priority and priority_level columns with the following case: if priority = 'High' then priority_level = 'Urgent'; if priority = 'Medium' then priority_level = 'Normal'; else priority_level = 'Low Priority'.

```
-- Categorize tasks by priority level ('Urgent' or 'Normal')
SELECT task_name, priority,
       CASE
         WHEN priority = 'High' THEN 'Urgent'
         WHEN priority = 'Medium' THEN 'Normal'
         ELSE 'Low Priority'
       END AS priority_level
FROM Tasks;
```

DQL operation using CASE
WHEN condition



	task_name	priority	priority_level
▶	Design database schema	High	Urgent
	Develop user login page	Medium	Normal
	Write test cases for login	Medium	Normal
	Create wireframes for homepage	High	Urgent
	Setup mobile app repository	High	Urgent
	Develop funds transfer module	High	Urgent
	Test funds transfer security	High	Urgent
	Install CMS software	Medium	Normal
	Create user documentation page	Low	Low Priority
	Analyze legacy database	High	Urgent
	Develop ETL scripts	High	Urgent
	API for stock update	High	Urgent
	Design inventory dashboard	Medium	Normal
	Finalize new logo design	Medium	Normal
	Implement new CSS styles	Medium	Normal
	Optimize product search query	High	Urgent
	Refactor authentication code	Medium	Normal
	Setup cloud database instance	High	Urgent
	Unassigned bug fix	Low	Low Priority
	Code Review - Security Audit	High	Urgent
	Code Review - Security Audit	High	Urgent

Result table showcasing
categorised tasks by priority level
(‘Urgent’ or ‘Normal’)

Data Query (DQL)

Subqueries (SINGLE-ROW)

From the Tasks table, the query displays the task_name and project_id columns, where the task_id is 10.

```
-- (Single-row) Find tasks that are in the same project as task_id 10
SELECT task_name, project_id
FROM Tasks
WHERE project_id = (
  SELECT project_id
  FROM Tasks
  WHERE task_id = 10
);
```

DQL operation demonstrating a single-row subquery



	task_name	priority	priority_level
▶	Design database schema	High	Urgent
	Develop user login page	Medium	Normal
	Write test cases for login	Medium	Normal
	Create wireframes for homepage	High	Urgent
	Setup mobile app repository	High	Urgent
	Develop funds transfer module	High	Urgent
	Test funds transfer security	High	Urgent
	Install CMS software	Medium	Normal
	Create user documentation page	Low	Low Priority
	Analyze legacy database	High	Urgent
	Develop ETL scripts	High	Urgent
	API for stock update	High	Urgent
	Design inventory dashboard	Medium	Normal
	Finalize new logo design	Medium	Normal
	Implement new CSS styles	Medium	Normal
	Optimize product search query	High	Urgent
	Refactor authentication code	Medium	Normal
	Setup cloud database instance	High	Urgent
	Unassigned bug fix	Low	Low Priority
	Code Review - Security Audit	High	Urgent
	Code Review - Security Audit	High	Urgent

Result table showcasing tasks that are in the same project as task_id 10

Data Query (DQL)

Subqueries (MULTIPLE-ROW)

From the Projects table, the query displays the first_name, last_name and job_title columns, where the employees are not project managers. The SELECT DISTINCT function helps to ensure each manager_id only appears once.

```
-- (Multiple-row - NOT IN) Find employees who are NOT project managers
SELECT first_name, last_name, job_title
FROM Employees
WHERE emp_id NOT IN (
    SELECT DISTINCT manager_id
    FROM Projects
    WHERE manager_id IS NOT NULL
);
```

DQL operation demonstrating a multiple-row subquery



	first_name	last_name	job_title
▶	Neo	Li Xin	Senior Developer
	Choh	Jing Yi	Junior Developer
	Lau	Ya Kai	QA Analyst
	Yap	Deh Kai	UI/UX Designer
	Chong	Joe Lim	Database Admin
	Kristen Chang	Jing Huan	Senior Developer
	Javier	Kastilanos	Junior Developer
	Jia	Li	QA Analyst
	Brendan	Chia	Intern
	Howard	Switsland	Business Analyst

Result table showcasing employees who are NOT 'Project Manager'

Data Query (DQL)

Subqueries (CORRELATED)

From the Projects table, the query displays the project_name column, where the Projects project_id is the same as Tasks project_id and has at least one 'High' priority task.

```
-- (Correlated) Find projects that have at least one 'High' priority task
SELECT p.project_name
FROM Projects p
WHERE EXISTS (
    SELECT 1
    FROM Tasks t
    WHERE t.project_id = p.project_id AND t.priority = 'High'
);
```

DQL operation demonstrating a correlated subquery



	project_name
▶	E-commerce Platform
	Mobile Banking App
	Data Warehouse Migration
	Inventory Management Tool

Result table showcasing projects that have at least one 'High' priority task

Data Query (DQL)

Set operations (UNION)

From the Employees and Projects table, the query displays the first_name column, where the lists consist of Employees first_name and Projects project_name using the UNION function.

```
-- (Multiple-row - NOT IN) Find employees who are NOT project managers
SELECT first_name, last_name, job_title
FROM Employees
WHERE emp_id NOT IN (
    SELECT DISTINCT manager_id
    FROM Projects
    WHERE manager_id IS NOT NULL
);
```

DQL operation using UNION operator



	first_name
▶	Chew
	Neo
	Choh
	Lau
	Yap
	Ramah
	Chong
	Kristen Chang
	Javier
	Jia

	Brendan
	Howard
	AI Chatbot
	Customer Feedback System
	Data Warehouse Migration
	E-commerce Platform
	HR Payroll System
	Internal Wiki
	Inventory Management Tool
	Mobile Banking App
	Website Redesign

Result table showcasing a list of employee and project names in a single column

Data Query (DQL)

Set operations (NOT EXISTS)

From the Employees table, the query will display the first_name and last_name columns, who do not have any assigned tasks.

```
-- Find employees with ZERO assigned tasks
SELECT e.first_name, e.last_name
FROM Employees e
WHERE NOT EXISTS (
    SELECT 1
    FROM Tasks t
    WHERE t.assignee_id = e.emp_id
);
```

DQL operation using NOT
EXISTS operator



	first_name	last_name
▶	Chew	Chiu Xian
	Ramah	bin Jaafar
	Brendan	Chia
	Howard	Switsland

Result table showcasing
employees with no assigned tasks

Data Query (DQL)

Joins (NATURAL JOIN)

From the Employees and Tasks table, the query will display all the columns from the two tables using the NATURAL JOIN function. This operator is not usually used because of its unpredictable behaviour and inconsistent table results.

```
-- (NATURAL JOIN) List all employees and the tasks they have
SELECT *
FROM Employees
NATURAL JOIN Tasks
LIMIT 10
```

DQL operation using
NATURAL JOIN operator



	emp_id	first_name	last_name	email	job_title	hire_date	task_id	project_id	assignee_id	task_name	status	priority
▶	12	Howard	Switsland	lena.schmidt@comp.com	Business Analyst	2022-04-10	1	1	2	Design database schema	Done	High
	11	Brendan	Chia	kumar.nair@comp.com	Intern	2024-06-01	1	1	2	Design database schema	Done	High
	10	Jia	Li	jia.li@comp.com	QA Analyst	2023-08-05	1	1	2	Design database schema	Done	High
	9	Javier	Kastlanos	ian.gomez@comp.com	Junior Developer	2024-01-10	1	1	2	Design database schema	Done	High
	8	Kristen Chang	Jing Huan	hana.chen@comp.com	Senior Developer	2022-03-16	1	1	2	Design database schema	Done	High
	7	Chong	Joe Lim	gani.i@comp.com	Database Admin	2023-01-30	1	1	2	Design database schema	Done	High
	6	Ramah	bin Jaafar	fara.ahmed@comp.com	Project Manager	2022-02-14	1	1	2	Design database schema	Done	High
	5	Yap	Deh Kai	evan.wong@comp.com	UI/UX Designer	2022-09-11	1	1	2	Design database schema	Done	High
	4	Lau	Ya Kai	diana.ravi@comp.com	QA Analyst	2022-07-01	1	1	2	Design database schema	Done	High
	3	Choh	Jing Yi	chris.lee@comp.com	Junior Developer	2023-05-20	1	1	2	Design database schema	Done	High

Result table of 2 tables joined
(Employees and Tasks)

Data Query (DQL)

Joins (INNER JOIN)

From the Employees, Tasks and Projects table, the query display the first_name, last_name, task_name, status and project_name columns, where the Employee emp_id is equal to Tasks assignee_id, Tasks project_id is equal to Projects project_id and ordered by their first name.

```
-- (INNER JOIN - 3 Tables) Get a full report: Employee, Task, and Project
SELECT
  e.first_name,
  e.last_name,
  t.task_name,
  t.status,
  p.project_name
FROM Employees e
JOIN Tasks t ON e.emp_id = t.assignee_id
JOIN Projects p ON t.project_id = p.project_id
ORDER BY e.first_name;
```

DQL operation demonstrating
INNER JOIN operation



first_name	last_name	task_name	status	project_name
Choh	Jing Yi	Develop user login page	In Progress	E-commerce Platform
Choh	Jing Yi	Create user documentation page	To Do	Internal Wiki
Chong	Joe Lim	Install CMS software	Done	Internal Wiki
Chong	Joe Lim	Analyze legacy database	In Progress	Data Warehouse Migration
Chong	Joe Lim	Setup cloud database instance	In Progress	Data Warehouse Migration
Javier	Kastlanos	Develop funds transfer module	In Progress	Mobile Banking App
Javier	Kastlanos	Implement new CSS styles	To Do	Website Redesign
Jia	Li	Test funds transfer security	In Progress	Mobile Banking App
Kristen Chang	Jing Huan	Setup mobile app repository	Done	Mobile Banking App
Kristen Chang	Jing Huan	Develop ETL scripts	In Progress	Data Warehouse Migration
Kristen Chang	Jing Huan	Refactor authentication code	To Do	Mobile Banking App
Kristen Chang	Jing Huan	Code Review - Security Audit	To Do	E-commerce Platform
Lau	Ya Kai	Write test cases for login	In Progress	E-commerce Platform
Neo	Li Xin	Design database schema	Done	E-commerce Platform
Neo	Li Xin	API for stock update	To Do	Inventory Management Tool
Neo	Li Xin	Optimize product search query	To Do	E-commerce Platform
Neo	Li Xin	Code Review - Security Audit	To Do	E-commerce Platform
Yap	Deh Kai	Create wireframes for homepage	Done	E-commerce Platform
Yap	Deh Kai	Design inventory dashboard	In Progress	Inventory Management Tool
Yap	Deh Kai	Finalize new logo design	Done	Website Redesign

Result table of 3 tables joined
(Employees, Tasks and Projects)

Data Query (DQL)

Joins (LEFT OUTER JOIN)

From the Tasks and Employees table, the query displays the first_name, last_name and task_count columns, where Employees emp_id is joined with Tasks assignee_id using LEFT JOIN operator, and descendingly ordered by task_count.

```
-- (LEFT OUTER JOIN) List ALL employees and the number of tasks they have
-- (Employees with 0 tasks will show 0)
SELECT
    e.first_name,
    e.last_name,
    COUNT(t.task_id) AS task_count
FROM Employees e
LEFT JOIN Tasks t ON e.emp_id = t.assignee_id
GROUP BY e.emp_id, e.first_name, e.last_name
ORDER BY task_count DESC;
```

DQL operation demonstrating
LEFT OUTER JOIN operation
using LEFT JOIN operator



	first_name	last_name	task_count
►	Neo	Li Xin	4
	Kristen Chang	Jing Huan	4
	Yap	Deh Kai	3
	Chong	Joe Lim	3
	Choh	Jing Yi	2
	Javier	Kastilanos	2
	Lau	Ya Kai	1
	Jia	Li	1
	Chew	Chiu Xian	0
	Ramah	bin Jaafar	0
	Brendan	Chia	0
	Howard	Switsland	0

Result table showcasing all
employees and their number of
tasks

Data Query (DQL)

Joins (SELF JOIN)

From the Employees table, the query displays the emp1_first, emp1_last, emp2_first, emp2_last and job_title columns, where the employees with the same job are paired together.

```
-- (SELF JOIN) Find pairs of employees who have the same job title
SELECT
  a.first_name AS emp1_first, a.last_name AS emp1_last,
  b.first_name AS emp2_first, b.last_name AS emp2_last,
  a.job_title
FROM Employees a
JOIN Employees b ON a.job_title = b.job_title AND a.emp_id < b.emp_id
ORDER BY a.job_title;
```

DQL operation demonstrating
SELF JOIN operation



	emp1_first	emp1_last	emp2_first	emp2_last	job_title
▶	Choh	Jing Yi	Javier	Kastilanos	Junior Developer
	Chew	Chiu Xian	Ramah	bin Jaafar	Project Manager
	Lau	Ya Kai	Jia	Li	QA Analyst
	Neo	Li Xin	Kristen Chang	Jing Huan	Senior Developer

Result table showcasing pairs of
employees who have the same job
title

BTREE Index

```
-- Create the BTREE index (MySQL's default)
CREATE INDEX idx_btree_hire_date
ON Employees(hire_date);

-- Run this AFTER creating the index to see the plan change.
-- It should now use the index ("idx_btree_hire_date")
EXPLAIN SELECT *
FROM Employees
WHERE hire_date BETWEEN '2022-01-01' AND '2022-12-31';
```

A tree-based data structure that allows for efficient data retrieval.

Popular in equality and range searches, but decent in ordering and prefix searches

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	Employees	NULL	ALL	NULL	NULL	NULL	NULL	12	11.11	Using where



	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	Employees	NULL	range	idx_btree_hire_date	idx_btree_hire_date	3	NULL	7	100.00	Using index condition

TEXT Index

```
-- Create the FULLTEXT index
CREATE FULLTEXT INDEX idx_text_description
ON Projects(project_description);

-- Query (After): Use the MATCH...AGAINST syntax.
-- Run this AFTER the index. It will show a "Full-text search".
EXPLAIN SELECT project_id, project_description
FROM Projects
WHERE MATCH(project_description) AGAINST('database' IN NATURAL LANGUAGE MODE);
```

Designed specifically for text content searching

Excel in searching through large text content and natural language search

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	Projects	NULL	ALL	NULL	NULL	NULL	NULL	9	11.11	Using where



	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	Projects	NULL	fulltext	idx_text_description	idx_text_description	0	const	1	100.00	Using where; Ft_hints: sorted

Team roles



CHOH JING YI
GROUP LEADER

Responsible for the overall ERD design, schema creation (**01_Schema.sql**), and final report compilation.



NEO LI XIN
QUERY SPECIALIST

Responsible for developing all **SELECT** queries, including joins, subqueries, aggregations, and functions (**03_Queries.sql**).
Responsible for **Index** by implementing **BTREE** and **TEXT** indexes.



CHEW CHIU XIAN
DATA MANAGER

Responsible for populating all tables with data (**02_Data.sql**) and performing the EXPLAIN analysis and indexing.

CONCLUSION

Challenges: Referential Integrity



Referential Integrity

Encountered errors deleting parent records in the Project table.
Solution: Must delete child records (Tasks) before parent records (Projects) to maintain integrity.

Challenges: Complex Queries



3-Table Joins

Linked Employees, Tasks, and Projects using INNER JOIN.
Solution: Applied strict logic to ON clauses for emp_id, assignee_id, and project_id to prevent missing data.

Improvement: Automation



Stored Procedures

Proposed to automate repetitive workflows.
Example: 'Onboard New Project' procedure to auto-insert projects and assign default tasks.

Improvement: Efficiency



Database Triggers

Proposed for automatic status updates to reduce manual work.
Example: Auto-set Project to 'Completed' when all associated Tasks are marked 'Done'.

THANK YOU



univteknologimalaysia



utm.my



utmofficial